

기존 도커 애플리케이션 삭제	\$ sudo apt remove docker docker-engine docker.io containerd runc
기존에 저장된 도커 오브젝트 (images, containers, volumes, network) 제거	\$ sudo rm -rf /var/lib/docker \$ sudo rm -rf /var/lib/containerd
도커 설치 스크립트 다운로드	\$ sudo apt-get update \$ sudo apt-get install curl \$ curl https://get.docker.com > docker-install.sh \$ chmod 755 docker-install.sh
도커 설치	\$ sudo ./docker-install.sh
우분투 컨테이너 생성	\$ sudo docker run -i -t ubuntu:14.04 -i : 상호 입출력 하겠다고 설정 -t : tty를 활성화해서 bash 셸을 사용하도록 설정
컨테이너 나가기 및 종료	\$ exit 또는 Ctrl + D : 배시셸을 종료함으로써 컨테이너를 정지 Ctrl + P 다음에 Ctrl + Q: 단순히 컨테이너의 셸만 빠져나온다.
도커 이미지 내려받기	\$ sudo docker pull centos:7
도커 이미지 추출	도커 이미지를 한 개의 파일로 추출하기 \$ sudo docker save -o ubuntu_14_04.tar ubuntu:14.04 도커 이미지 로드하기 \$ sudo docker load -i ubuntu_14_04.tar
도커 이미지 다루기	도커 허브라는 중앙 이미지 저장소에서 도커 이미지 검색하기 \$ sudo docker search 키워드`
도커 이미지 생성	이미지 목록 조회 \$ sudo docker images 이미지로 만들 컨테이너 준비 \$ sudo docker run -i -t --name commit_test ubuntu:14.04 컨테이너 변경: `# echo test_first! >> first` 컨테이너를 이미지로 만들기 \$ sudo docker commit 옵션 컨테이너명 REPOSITORY:TAG \$ sudo docker commit -a "alicek106" -m "my first commit" commit_test commit_test:first -a author : 이미지 작성자에 대한 정보를 이미지에 포함시킨다. -m 커밋메시지 : 이미지에 포함될 부가 설명을 설정한다. 만든 이미지로 컨테이너 만들기 \$ sudo docker run -i -t --name commit_test2 commit_test:first 컨테이너 변경: `# echo test_second! >> second` 변경한 컨테이너로 새 이미지 만들기 \$ sudo docker commit -a "alicek106" -m "my second commit" commit_test2 commit_test:second`
도커 이미지 목록 확인하기	\$ sudo docker images
도커 이미지 정보 조회	이미지 정보 살펴보기 \$ sudo docker inspect ubuntu:14.04 \$ sudo docker inspect commit_test:first Layers = ubuntu:14.04 Layers + 추가사항 I \$ sudo docker inspect commit_test:second Layers = commit_test:first Layers + 추가사항 II Layers = ubuntu:14.04 Layers + 추가사항 I + 추가사항 II 이미지 레이어 구조에 대한 변경 내역 확인하기 \$ sudo docker history 도커이미지명 \$ sudo docker history commit_test:first
컨테이너 기본 명령어 모음	sudo docker stop miracle 중지 sudo docker restart miracle 재부팅
도커 이미지로 컨테이너 생성하기	\$ sudo docker create -i -t --name mycentos centos:7
도커 컨테이너 추가하기	\$ sudo docker run -i -t --name miracle ubuntu /bin/bash
도커 컨테이너 추가 및 종료해도 백그라운드로 실행되는 명령어	\$ sudo docker run -i -t -d --name miracle ubuntu /bin/bash
도커 무작위이름 새이름	\$ sudo docker rename
도커 컨테이너 실행하기	\$ sudo docker start name \$ sudo docker start 해시아이디일부분
도커 컨테이너에 들어가기	\$ sudo docker attach name
도커 실행	\$ sudo run
도커 실행 중인 컨테이너 목록 보기	\$ sudo docker ps
모든 컨테이너 목록 보기	\$ sudo docker ps -a
컨테이너 ID 전체 보기	\$ sudo docker inspect 컨테이너이름 grep Id

도커 상태 설명	- CONTAINER ID: 컨테이너에 자동 할당되는 고유한 ID. - IMAGE: 컨테이너 이미지 이름 - COMMAND: 컨테이너가 시작될 때 실행될 명령 - CREATED: 컨테이너가 생성된 후 흐른 시간 - STATUS: 컨테이너 상태. Up(실행중), Exited(종료), Pause(중지) - PORTS: 컨테이너가 개방한 포트와 호스트에 연결한 포트. 외부에 노출하도록 설정하지 않았다면 출력 내용 없음. - NAMES: 컨테이너의 고유한 이름. --name 옵션으로 설정한 이름. 설정하지 않으면 형용사와 명사를 사용해 무작위 생성.
사용하지 않는 컨테이너 삭제하기	\$ sudo docker rm epic_dubinsky
실행 중인 컨테이너 삭제하기	\$ sudo docker stop mycentos : 실행 중인 컨테이너 정지시키기 \$ sudo docker rm mycentos : 컨테이너 삭제하기 또는 -f 옵션을 사용하여 종료와 삭제를 한 번에 처리하기 ex): \$ sudo docker rm -f mycentos
정지된 모든 컨테이너 삭제하기	\$ sudo docker container prune
실행 중인 컨테이너 모두 정지 후 삭제하기	\$sudo docker stop \$(sudo docker ps -a -q) \$ sudo docker rm \$(sudo docker ps -a -q)
도커 이미지 삭제	이미지 삭제하기 \$ sudo docker rmi commit_test:first 이미지를 사용 중인 컨테이너가 있을 경우 삭제할 수 없다. 컨테이너를 먼저 삭제한 후 이미지를 삭제해야 한다. \$ sudo docker stop commit_test2 && sudo docker rm commit_test2 땀글렁(dangling) 이미지 다루기 컨테이너 생성 \$ sudo docker run -i -t --name dangle_test commit_test:second 컨테이너 확인 \$ sudo docker ps -a 컨테이너가 사용한 이미지 강제 삭제 \$ sudo docker rmi -f commit_test:second 컨테이너 확인 \$ sudo docker ps -a 이미지 이름이 변경되어 있다. 삭제된 이미지 확인 \$ sudo docker images -f dangling=true
도커가 설치된 호스트에서만 접근 가능	\$ sudo docker run -i -t --name network_test ubuntu:14.04 - NIC 확인: # ifconfig
A컨테이너와 B컨테이너 준비	\$ sudo docker run --name B_Container -d B \$ sudo docker run --name A_Container -d -p 80:80 --link B_Container:B_container A
컨테이너를 노출 시키기	\$ sudo docker run -i -t --name mywebserver -p 80:80 ubuntu:14.04' -p 호스트포트:컨테이너포트 여러 개의 포트 노출: '-p' 옵션을 여러 개 삽입
컨테이너에 아파치 웹 서버 설치 및 시작시키기	\$sudo apt-get update \$sudo apt-get install apache2 -y \$sudo service apache2 start
컨테이너를 실행하고 있는 호스트로 접속하기	http://vagrant리눅스IP/vagrant ssh 밖에서 실행할 것
이미지와 컨테이너의 세부 정보	\$sudo sudo docker inspect 컨테이너이름 or 이미지 이름
호스트와 바인딩된 포트번호 확인하기	\$ sudo docker port 컨테이너이름 \$ sudo docker port mywebserver
Detached 모드 컨테이너의 내부 셸을 사용하기	상호 입출력 가능한 상태로 접속하기 \$ sudo docker exec -i -t 컨테이너이름 /bin/bash' 컨테이너 내부의 실행 결과만 확인하기 \$ sudo docker exec 컨테이너이름 ls
도커 볼륨 다루기 호스트 볼륨 공유하기	\$ sudo docker run -d --name wordpresddb_hostvolume -e MYSQL_ROOT_PASSWORD=password -e MYSQL_DATABASE=wordpress -v /home/wordpress_db:/var/lib/mysql mysql:5.7 - v 호스트의공유디렉토리:컨테이너의공유디렉토리 - 호스트의 공유 디렉토리가 없으면 도커가 자동 생성됨 \$ sudo docker run -d --name wordpress_hostvolume -e WORDPRESS_DB_HOST=mysql -e WORDPRESS_DB_USER=root -e WORDPRESS_DB_PASSWORD=password --link wordpresddb_hostvolume:mysql -p 80 wordpress 이미지를 생성할때부터 공유 폴더 지정 docker run -i -t --name 컨테이너이름 -v /data 이미지이름 디렉토리를 설정해서 이미지를 생성할때부터 공유 폴더 지정 docker run -it --name 컨테이너이름 -v 호스트 주소:컨테이너 주소 이미지이름 ex) docker run -it --name se01 -v /usr/local/apache/share:/data ubuntu docker run -it --name se02 -v /usr/local/apache/share:/data ubuntu docker run -it --name se03 -v /usr/local/apache/share:/data ubuntu

볼륨 컨테이너 공유하기	<pre>volume_override 컨테이너의 볼륨을 공유하기 \$ sudo docker run -i -t --name volumes_from_container --volumes-from volume_override ubuntu:14.04` 컨테이너에 추가된 디렉토리 확인: `# ls /home/testdir_2/`</pre>
컨테이너 삭제 후에도 데이터가 보존되는 것을 확인하기	<pre>\$ sudo docker stop wordpress_hostvolume wordpressdb_hostvolume \$ sudo docker rm wordpress_hostvolume wordpressdb_hostvolume</pre>
호스트 디렉토리를 컨테이너의 존재하는 디렉토리와 연결할 수 있다.	<pre>\$ sudo docker run -i -t --name volume_dummy alicek106/volume_test 컨테이너에 존재하는 디렉토리 확인: `# ls /home/testdir_2/` \$ sudo docker run -i -t --name volume_override -v /home/wordpress_db:/home/testdir_2 alicek106/volume_test 컨테이너에 존재하는 디렉토리 확인: `# ls /home/testdir_2/` 기존의 디렉토리를 호스트 디렉토리로 대체한다.</pre>
도커 볼륨 사용하기	<pre>도커 볼륨 생성하기 \$ sudo docker volume create --name myvolume 도커 볼륨 조회하기 \$ sudo docker volume ls 도커 볼륨 사용하기 \$ sudo docker run -i -t --name myvolume_1 -v myvolume:/root/ ubuntu:14.04 # echo Hello, volume! >> /root/volume`  도커 볼륨 공유하기 \$ sudo docker run -i -t --name myvolume_2 -v myvolume:/root/ ubuntu:14.04 # cat /root/volume` 도커 볼륨의 실제 위치 알아내기 \$ sudo docker inspect --type volume myvolume docker inspect` 명령: 컨테이너, 이미지, 볼륨 등 도커의 모든 구성 단위의 정보를 확인할 때 사용 type [image volume container] 도커 볼륨 자동 생성하기 \$ sudo docker run -i -t --name volume_auto -v /root/ ubuntu:14.04 컨테이너가 사용하는 도커 볼륨 확인하기 \$ sudo docker container inspect volume_auto 사용하지 않는 볼륨을 자동으로 삭제하기 \$ sudo docker volume prune</pre>
mount 옵션	<pre>mount 옵션으로 도커 볼륨 연결하기 \$ sudo docker run -i -t --name mount_option_1 --mount type=volume,source=myvolume,target=/root/ ubuntu:14.04 mount 옵션으로 호스트 디렉토리를 컨테이너에 연결하기 \$ sudo docker run -i -t --name mount_option_2 --mount type=bind,source=/home/wordpress_db,target=/root/ ubuntu:14.04</pre>
도커 네트워크	<pre>도커에서 기본적으로 쓸 수 있는 네트워크 확인하기 \$ sudo docker network ls bridge: 컨테이너를 생성할 때 자동으로 연결되는 docker0 브리지를 활용하도록 설정됨 172.17.0.x IP 대역을 컨테이너에 순차적으로 할당한다.</pre>
도커 네트워크 다루기	<pre>도커 호스트의 가상 이더넷 카드 \$ ifconfig - 실행 중인 컨테이너 개수 만큼 `vethxxxx` 가상 이더넷 카드가 생성된 것을 확인 할 수 있다.</pre>
워드프레스 기반 블로그 서비스 만들기	<pre>\$ sudo docker run -d --name wordpress -e WORDPRESS_DB_HOST=mysql -e WORDPRESS_DB_USER=root -e WORDPRESS_DB_PASSWORD=password --link wordpressdb:mysql -p 80 wordpress</pre>
접속 확인	<pre>http://vagrant리눅스IP:자동설정된포트번호/ - vagrant ssh 밖에서 실행할 것</pre>
도커 컨테이너 run 옵션	<pre>\$ sudo docker run -d ... -e ... --link ... -d : Detached 모드로 컨테이너 실행. 컨테이너에서 백그라운드에서 동작하는 애플리케이션을 실행할 때 사용. 이 모드에서는 컨테이너가 실행하는 프로그램이 없으면 자동 종료된다. 테스트 \$ sudo docker run -d --name detach_test ubuntu:14.04 \$ sudo docker ps -a 로 확인해보면 컨테이너가 실행 즉시 종료되었음을 확인 가능 - e 환경변수명=값 : 컨테이너 내부의 환경변수 설정. 컨테이너에서 실행되는 애플리케이션이 이 환경 변수를 사용 - Detached 모드 컨테이너의 내부 환경 변수 확인 셸 접속: \$ sudo docker exec -i -t wordpressdb /bin/bash - 환경 변수 확인: `# echo \$MYSQL_ROOT_PASSWORD` --link 컨테이너명:별명` : 내부 IP를 알 필요 없이 컨테이너 별명으로 접근하도록 설정 - 도커 엔진은 컨테이너에게 내부 IP를 172.17.0.2, 3, 4, ... 와 같이 순차적으로 할당 테스트 \$ sudo docker exec wordpress curl mysql:3306 --silent</pre>

주의사항	도커 이미지를 다운로드 받을 때 플랫폼 이름을 명시하지 않으면 현재 사용하는 OS에 맞는 이미지를 찾는다. macOS 플랫폼에 맞는 MySQL 이미지를 찾을 수 없다. macOS에서 이미지를 다운로드 받고 싶다면 플랫폼 이름을 명시 \$ sudo docker run --platform linux/amd64 -d --name wordpressdb -e MYSQL_ROOT_PASSWORD=password -e MYSQL_DATABASE=wordpress mysql:5.7
도커 허브에 로그인 하기	\$ sudo docker login
도커 이미지 배포	도커 허브(<https://hub.docker.com/>)에 회원 가입 및 로그인
저장소에 이미지 올리기	\$ sudo docker push new-repo:tagname \$ sudo docker push eomjinyoung/hello-docker:0.0 도커 허브 사이트의 Tags 탭에서 확인할 것!
저장소에 업로드한 이미지 가져오기	\$ sudo docker pull eomjinyoung/hello-docker:0.0
가져온 이미지로 컨테이너 생성, 실행 및 접속	도커 이미지 목록 확인하기 \$ sudo docker images 도커 이미지로 컨테이너 생성하기 \$ sudo docker create -i -t --name hello1 eomjinyoung/hello-docker:0.0 도커 컨테이너 실행하기 \$ sudo docker start hello1 도커 컨테이너에 들어가기 \$ sudo docker attach hello1
이미지 저장소 생성	Create a Repository_ 클릭 저장소명: hello-docker 설명: 테스트용 Visibility: private
테스트용 이미지 생성	\$ sudo docker run -i -t --name commit-container1 ubuntu:14.04 컨테이너 변경: # echo my first push >> test
테스트용 이미지 커밋하기	\$ sudo docker commit commit-container1 hello-docker:0.0
이미지에 태깅하기	\$ sudo docker tag local-image:tagname new-repo:tagname \$ sudo docker tag hello-docker:0.0 eomjinyoung/hello-docker:0.0
웹 애플리케이션 배포 - 직접 컨테이너 구성 및 배치하기 MariaDB 컨테이너 생성	MariaDB 도커 이미지 가져오기 \$ sudo docker pull mariadb MariaDB 컨테이너 생성 및 실행 \$ sudo docker run -p 3306:3306 --detach --name mariadb --env MARIADB_USER=study --env MARIADB_PASSWORD=1111 --env MARIADB_ROOT_PASSWORD=1111 mariadb MariaDB 컨테이너 접속 \$ sudo docker exec -it mariadb /bin/bash MariaDB 버전 확인 # mysql --version MariaDB 클라이언트 실행 및 root 사용자로 서버 접속 # mysql -u root -p 웹 애플리케이션의 DB 환경 구축 - studydb 데이터베이스 생성 - CREATE DATABASE studydb CHARACTER SET utf8 COLLATE utf8_general_ci - study 사용자에게 'studydb' 데이터베이스 사용권한 부여 - GRANT ALL ON studydb.* TO 'study'@'%' - study 사용자로 로그인 # mysql -u study -p study 사용자가 사용할 수 있는 데이터베이스 확인 show databases 웹 애플리케이션 테이블 생성 - myapp/app-server/doc/ - ddl.sql 실행 - ddl2.sql 실행 - ddl3.sql 실행 - ddl4.sql 실행 - data4.sql 실행
ubuntu 에서 mysql 명령 실행하려면 설치 명령어	\$sudo apt install mariadb-server mariadb-client -y

myapp 웹 애플리케이션 컨테이너 생성 환경변수 설치	<pre>Ubuntu 22.04 컨테이너 생성 및 실행 \$ sudo docker run -p 80:80 -it --name myapp ubuntu ifconfig 등 네트워킹 관련 프로그램 추가 # apt update # apt install net-tools nano 에디터 설치 # apt install nano` JDK 17 설치 # apt install openjdk-17-jdk JAVA_HOME 환경 변수 설정 # echo 'export JAVA_HOME=/usr/lib/jvm/java-17-openjdk-amd64' tee /etc/profile.d/java17.sh # source /etc/profile.d/java17.sh 환경 변수 적용 # source /etc/profile.d/java17.sh git 설치 # apt install git wget 설치 # apt install wget unzip 설치 # apt install unzip ping 설치 # apt-get install iputils-ping gradle 다운로드 및 설치 # VERSION=8.0.2` # wget https://services.gradle.org/distributions/gradle-\${VERSION}-bin.zip -P /tmp # unzip -d /opt/gradle /tmp/gradle-\${VERSION}-bin.zip # ln -s /opt/gradle/gradle-\${VERSION} /opt/gradle/latest # echo 'export GRADLE_HOME=/opt/gradle/latest' tee /etc/profile.d/gradle.sh # echo 'export PATH=\${GRADLE_HOME}/bin:\${PATH}' tee -a /etc/profile.d/gradle.sh # source /etc/profile.d/gradle.sh myapp git 저장소 가져오기 # mkdir git # cd git # git clone https://github.com/eomjinyoung/bitcamp-study nodejs 및 npm 설치 # apt install nodejs # apt install npm myapp 자바스크립트 라이브러리 설치 # cd ~/git/bitcamp-study/myapp/app-server/src/main/webapp ~/git/bitcamp-study/myapp/app-server/src/main/webapp# npm install myapp 빌드 # cd ~/git/bitcamp-study/myapp ~/git/bitcamp-study/myapp# gradle build ping 설치 # apt-get install iputils-ping -y mariadb 접속 확인 # ping 172.17.0.2 myapp 실행 ~# java -jar ~/git/bitcamp-study/myapp/app-server/build/libs/app-server-0.0.1-SNAPSHOT.jar</pre>
셸스크립트를 사용해서 docker_container에 명령 보내기	<pre>docker restart 컨테이너_이름 docker exec 컨테이너_이름 sh -c "도커 컨테이너에 보낼 명령문; 도커 컨테이너에 보낼 명령문"</pre>

Dockerfile 다루기 스프링부트 웹 프로젝트를 도커 컨테이너로 실행하기	FROM openjdk:17 ARG JAR_FILE=build/libs/app-server-0.0.1-SNAPSHOT.jar COPY \${JAR_FILE} app.jar ENTRYPOINT ["java", "-jar", "app.jar"]
Dockerfile로 이미지 생성하기	\$sudo docker build -t eomjinyoung/myapp:0.0.2 .
컨테이너 생성 및 실행하기	\$sudo docker run -d -p 80:80 -v /mnt/c/Users/bitcamp/webapp-upload:/root/webapp-upload --name myapp eomjinyoung/myapp:0.0.2
container에서 파일 꺼내기	docker cp <컨테이너 이름>:<경로> <호스트 경로> \$sudo docker cp myname:/data ./

!요: 기존에 Docker에 명령을 하기 위해서는 도커 커맨드라인 또는 쉘스크립트를 통해서 직접 명령을 수행 docker compose를 사용하면 쉽게 docker를 운용 가능	
Docker Compose 설치하기	curl -L "https://github.com/docker/compose/releases/download/1.9.0/docker-compose-\$(uname -s)-\$(uname -m)" -o /usr/local/bin/docker-compose chmod +x /usr/local/bin/docker-compose # test docker-compose version
docker-compose.yml을 통해서 Docker에 명령	docker-compose -f docker-compose-local.yml up Docker Compose 설정 파일 지정 -f옵션 사용
백그라운드에서 컨테이너 띄움	docker-compose up -d
Docker Compose에 정의되어 있는 모든 서비스를 한번에 정지	docker-compose down
Docker Compose에 정의되어 있는 모든 서비스 컨테이너 목록 조회	docker-compose ps
서비스 컨테이너의 로그를 확인	docker-compose logs
실행 중인 서비스 컨테이너를 대상으로 명령어 전송	docker-compose exec
서비스 컨테이너의 특정 명령어를 일회성으로 실행할 때 사용	docker-compose run
Docker Compose 설정을 확인할 때 사용	docker-compose config